SmartLender: ML-Powered Loan Approval Prediction System

Project Description:

SmartLender is a machine learning-powered web application designed to predict the creditworthiness of loan applicants, enabling banks and financial institutions to make faster, data-driven loan approval decisions. The platform leverages four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost (Gradient Boosting) — to evaluate applicant data and determine the likelihood of loan repayment or default.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models, the best-performing model (XGBoost at 94.7% training accuracy and 81.1% testing accuracy) is saved and integrated into a Flask web application for real-time prediction.

Built with Python and Flask and designed for deployment on Hugging Face Spaces (free cloud), SmartLender provides a seamless user interface where applicants can submit their details and instantly receive an approval prediction. The system empowers financial analysts, credit officers, and banking professionals to reduce non-performing assets and improve the overall credit evaluation process with confidence.

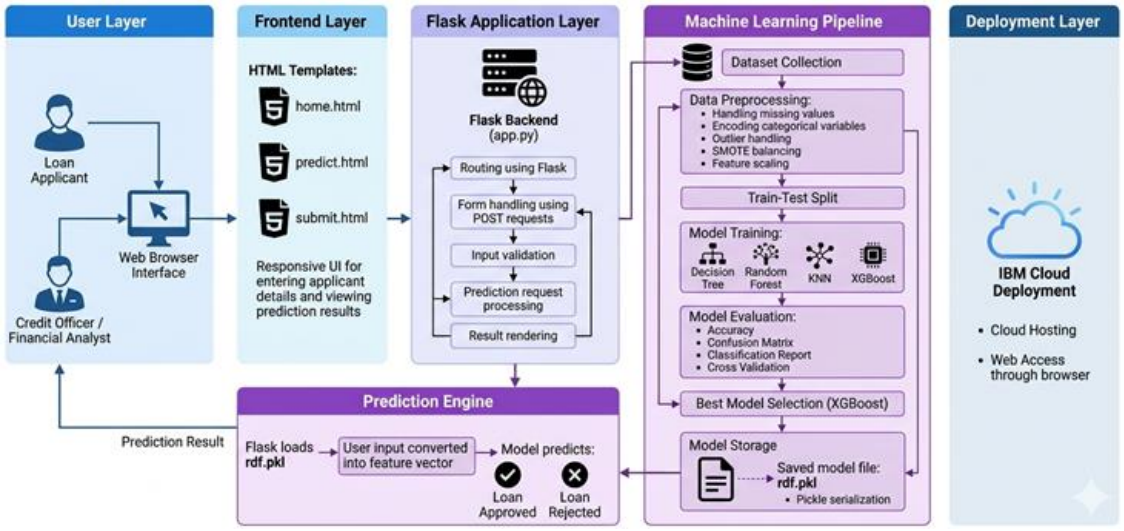
Scenarios

Scenario 1: A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

Scenario 2: An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk based on the trained model, prompting further scrutiny or document verification before proceeding.

Scenario 3: A financial analyst uses the platform to evaluate multiple applicants during a high-volume period. By relying on the XGBoost model's predictions, the analyst significantly reduces evaluation time while maintaining accuracy and compliance.

Technical Architecture:



Description: SmartLender utilizes a modular five-layer architecture to deliver rapid, ML-driven loan predictions. The User Layer allows loan applicants and credit officers to interact via a web browser. The Frontend Layer provides responsive HTML templates (home.html, input.html, output.html). The Flask Application Layer handles routing, form processing, input validation, and result rendering. The Machine Learning Pipeline covers data collection, preprocessing (SMOTE balancing, feature scaling), model training across four algorithms, best model selection (XGBoost), and model storage via pickle serialisation. Finally, the Deployment Layer hosts the application on Hugging Face Spaces via Docker for free, permanent cloud access.

(Note: The ER Diagram for SmartLender is included in the Database Design section as the system uses a relational data model with six entities: USER, APPLICANT_PROFILE, CREDIT_HISTORY, LOAN_APPLICATION, MODEL, and PREDICTION_RESULT.)

Pre-requisites:

- **Python Programming Proficiency:** https://www.python.org/doc/
- **Flask Framework Knowledge:** https://flask.palletsprojects.com/
- **Machine Learning with Scikit-learn:** https://scikit-learn.org/stable/
- **HTML, CSS Skills:** https://www.w3schools.com/html/
- **Pandas & NumPy for Data Processing:** https://pandas.pydata.org/docs/
- **Anaconda Navigator Setup:** https://www.anaconda.com/download
- **Docker for Deployment:** https://docs.docker.com/
- **Hugging Face Spaces Deployment:** https://huggingface.co/spaces

Project Workflow:

Milestone 1: Data Collection and Architecture Design

- **Activity 1.1:** Download the required dataset (loan_prediction.csv) from the provided source and store it in the Dataset/ project directory.
- **Activity 1.2:** Define the application architecture, including data flow, machine learning workflow, database design, and deployment structure.

Milestone 2: Visualising and Analysing the Data

- **Activity 2.1:** Import and read the dataset using Pandas for further analysis.
- **Activity 2.2:** Perform univariate analysis to understand the distribution of individual features using distplot and countplot.
- **Activity 2.3:** Conduct bivariate analysis to identify relationships between two variables using countplot with hue.
- **Activity 2.4:** Perform multivariate analysis to uncover patterns involving multiple features using swarmplot.

Milestone 3: Data Pre-Processing

- **Activity 3.1:** Check for missing values, duplicates, and inconsistencies, and handle them appropriately using mode and median imputation.
- **Activity 3.2:** Encode categorical variables (Gender, Married, Education, Self_Employed, Property_Area, Loan_Status) to numerical values.
- **Activity 3.3:** Balance the dataset using SMOTE (Synthetic Minority Over-sampling Technique) to ensure fair representation of all target classes.
- **Activity 3.4:** Scale and normalise numerical features using StandardScaler and split into training (67%) and testing (33%) sets.

Milestone 4: Model Building

- **Activity 4.1:** Train and evaluate a Decision Tree model on the prepared dataset.
- **Activity 4.2:** Build and test a Random Forest model to improve predictive performance.
- **Activity 4.3:** Implement a K-Nearest Neighbours (KNN) model and analyse its accuracy.
- **Activity 4.4:** Train an XGBoost (GradientBoostingClassifier) model and compare its performance with other models.
- **Activity 4.5:** Evaluate all models using 5-fold cross-validation and save the best-performing model (XGBoost) as rdf.pkl using pickle.

Milestone 5: Application Building

- **Activity 5.1:** Design and develop HTML pages (home.html, input.html, output.html) to create a user-friendly dark-themed interface.
- **Activity 5.2:** Build the Flask application (app.py) and integrate the trained XGBoost model and StandardScaler for real-time predictions.

- **Activity 5.3:** Run, test, and validate the application locally to ensure accurate predictions and smooth functionality.

Milestone 6: Deployment

- **Activity 6.1:** Containerise the application using Docker with a Dockerfile configured for port 7860.
- **Activity 6.2:** Deploy the application on Hugging Face Spaces for free, permanent cloud hosting accessible globally.

Milestone 7: Conclusion

Milestone 1: Data Collection and Architecture Design

In this milestone, we focus on collecting the loan prediction dataset and designing the complete application architecture. The dataset is sourced from a standard loan prediction challenge containing real-world applicant data. The architecture is designed to ensure separation of concerns, modularity, and scalability across all layers of the system.

Activity 1.1: Download and Setup the Dataset

The loan prediction dataset is available in CSV and Excel formats. Follow the steps below to set up the dataset in your project:

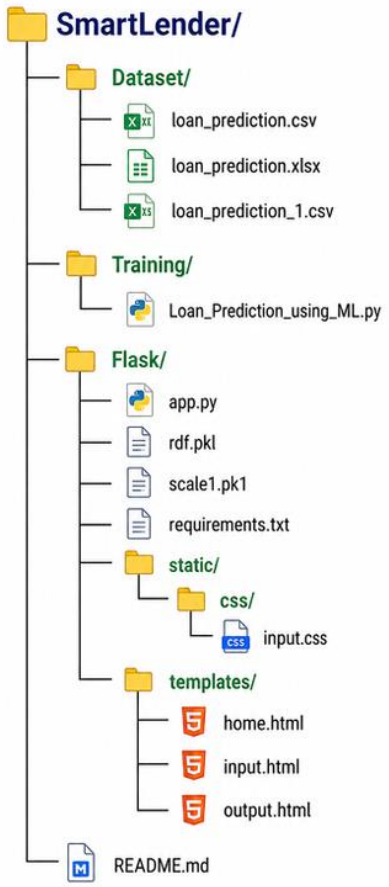
- **Step 1 —** Download the dataset file loan_prediction.csv from the provided source.
- **Step 2 —** Create a project folder named SmartLender/ on your local machine.
- **Step 3 —** Inside SmartLender/, create a Dataset/ subfolder and place all dataset files there.
- **Step 4 —** Verify the dataset by opening it in Excel or a text editor to confirm all 614 rows and 13 columns are present.

The dataset contains the following key information:

- **614 loan application records**
- **13 features including applicant demographics, income, loan details, and credit history**
- **Target variable: Loan_Status (Y = Approved, N = Rejected)**
- **Class distribution: approximately 68% approved, 32% rejected (imbalanced — addressed using SMOTE)**

Activity 1.2: Define the Application Architecture

A modular and scalable application architecture was designed with the following directory structure:



The project follows a clean three-tier structure:

- **Dataset/ —** Contains all raw data files (loan_prediction.csv, loan_prediction.xlsx, loan_prediction_1.csv)
- **Training/ —** Contains the complete ML pipeline script (Loan_Prediction_using_ML.py)
- **Flask/ —** Contains the web application: app.py, model files (rdf.pkl, scale1.pkl), requirements.txt, Dockerfile, templates/, and static/

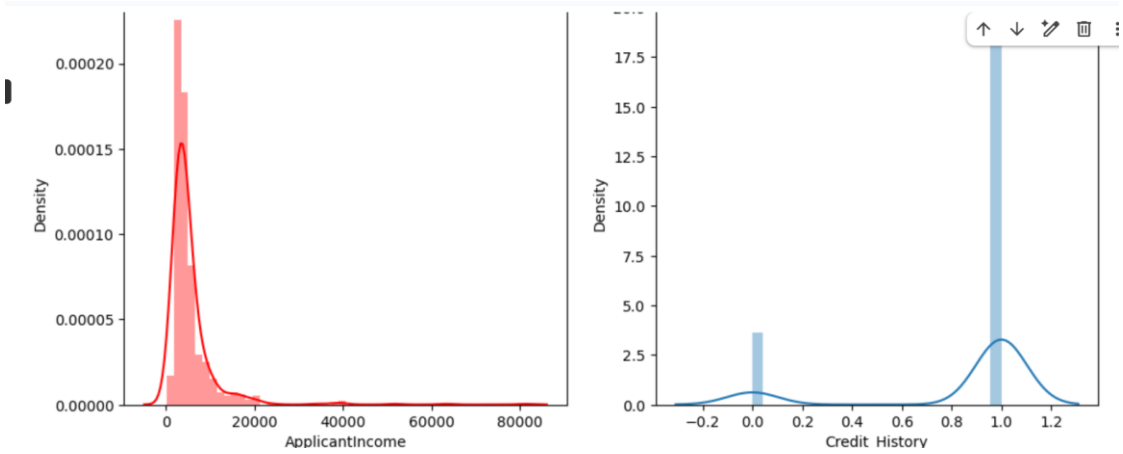
Milestone 2: Visualising and Analysing the Data

Milestone 2 focuses on understanding the dataset through comprehensive exploratory data analysis (EDA). By visualising individual features and their relationships, we identify patterns, distributions, and insights that guide the preprocessing and modelling decisions.

Activity 2.1: Import and Read the Dataset

The dataset is loaded using the Pandas library. The read_csv() function is used to load it by passing the file directory path as a parameter.

```
data = pd.read_csv('../Dataset/loan_prediction.csv')
data.head()
```

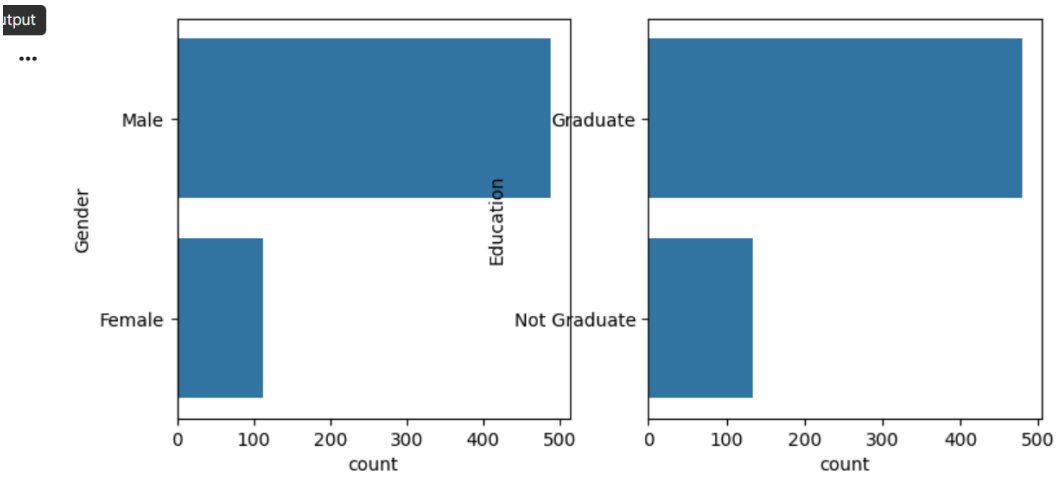


The dataset contains 614 rows and 13 columns including Loan_ID, Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area, and Loan_Status.

Activity 2.2: Univariate Analysis

Univariate analysis examines the distribution and characteristics of individual features in the dataset. Distribution plots (histplot/distplot) are used for continuous variables and count plots for categorical variables.

```
plt.figure(figsize=(12,5))
plt.subplot(121)
sns.distplot(data['ApplicantIncome'], color='r')
plt.subplot(122)
sns.distplot(data['Credit_History'])
plt.show()
```




```
#plotting the count plot
plt.figure(figsize=(18,4))
plt.subplot(1,4,1)
sns.countplot(data['Gender'])
plt.subplot(1,4,2)
sns.countplot(data['Education'])
plt.show()
```

Observations:

- **Applicant income** exhibits a right-skewed distribution — most applicants earn below ₹10,000/month with a few high-income outliers.
- **Credit history** is a binary feature with values 0 and 1. The majority of applicants (≈85%) have a good credit history (value = 1).

```
plt.figure(figsize=(18,4))
plt.subplot(1,4,1)
sns.countplot(data['Gender'])
plt.subplot(1,4,2)
sns.countplot(data['Education'])
plt.show()
```

```
[ ]
#visulaized based gender and income what would be the appplication status
sns.swarmplot(data['Gender'],data['ApplicantIncome'], hue = data['Loan_Status'])
```

```
#visualising two colunms against each other
plt.figure(figsize=(20,5))
plt.subplot(131)
sns.countplot(data['Married'], hue=data['Gender'])
plt.subplot(132)
sns.countplot(data['Self_Employed'], hue=data['Education'])
plt.subplot(133)
sns.countplot(data['Property_Area'], hue=data['Loan_Amount_Term'])
```

Observations:

- **Gender:** Male applicants (≈80%) significantly outnumber Female applicants (≈20%).
- **Education:** Graduate applicants (≈80%) dominate over Non-Graduate applicants (≈20%).

Activity 2.3: Bivariate Analysis

Bivariate analysis uses count plots to examine relationships between pairs of variables.

```
plt.figure(figsize=(20,5))
plt.subplot(131)
sns.countplot(data['Married'], hue=data['Gender'])
plt.subplot(132)
sns.countplot(data['Self_Employed'], hue=data['Education'])
plt.subplot(133)
sns.countplot(data['Property_Area'], hue=data['Loan_Amount_Term'])
plt.show()
```

Handling Categorical values

```
import jupyterthemes as jt
```

```
!jt -t onedork
```

```
data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban': 1,'Rural':0})
data['Married']=data['Married'].map({'Yes':1,'No':0})
data['Education']=data['Education'].map({'Graduate':1,'Not Graduate':0})
data['Loan_Status']=data['Loan_Status'].map({'Y':1,'N':0})
```

```
#Handling categorical feature Gender
data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data.head()
```

```
data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban': 1,'Rural':0})
```

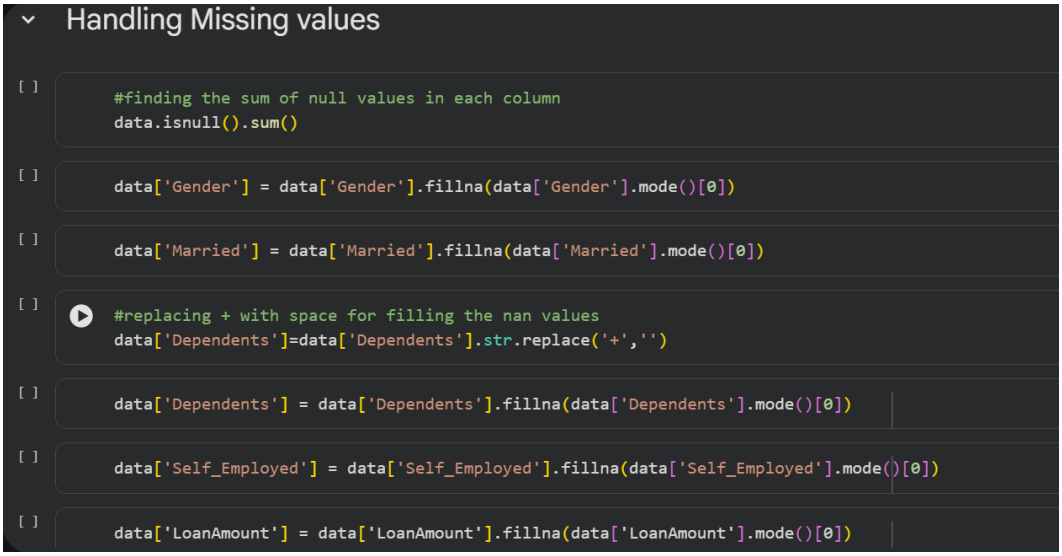
Observations from bivariate analysis:

- **Married vs Gender:** Married male applicants form the largest demographic group in the dataset.
- **Self_Employed vs Education:** Graduate applicants tend to be salaried employees rather than self-employed, indicating higher financial stability.
- **Property_Area vs Loan_Amount_Term:** Urban applicants dominate the standard 360-day loan term across all property areas.

Activity 2.4: Multivariate Analysis

Multivariate analysis examines the relationships between multiple features simultaneously using swarmplot.

```
sns.swarmplot(data['Gender'], data['ApplicantIncome'], hue=data['Loan_Status'])
```



Observation: Higher-income applicants across both genders tend to receive loan approval. Low-income applicants are more likely to be rejected regardless of gender, confirming that income is a key predictor of loan approval.

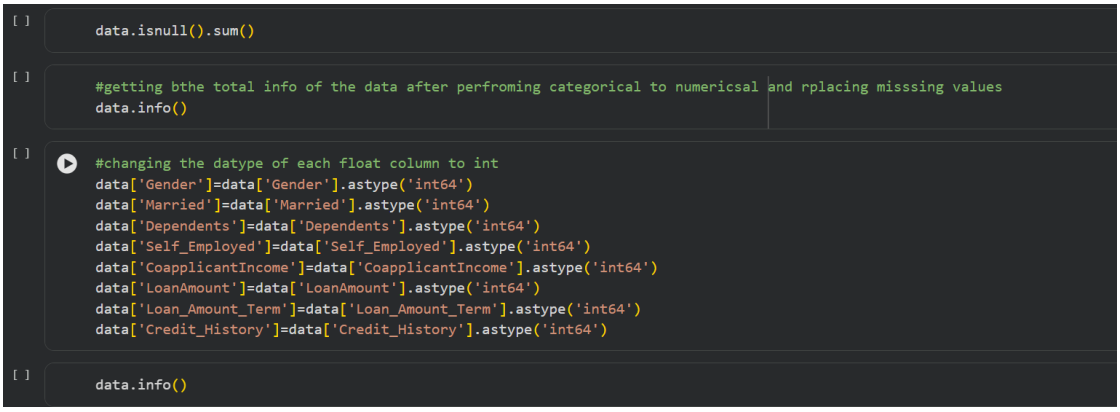
Milestone 3: Data Pre-Processing

Milestone 3 focuses on preparing the dataset for machine learning by handling missing values, encoding categorical variables, balancing classes, scaling features, and splitting the data into training and testing sets.

Activity 3.1: Handling Missing Values

Missing values were identified using data.isnull().sum() and treated as follows:

```
#finding the sum of null values in each column
data.isnull().sum()
```



```
data.isnull().sum()

#getting bthe total info of the data after perfroming categorical to numericsal and rplacing misssing values
data.info()

#changing the datype of each float column to int
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')

data.info()
```

- **Gender:** Filled with mode (Male) — 13 missing values
- **Married:** Filled with mode (Yes) — 3 missing values
- **Dependents:** "+" replaced with "" then filled with mode — 15 missing values
- **Self_Employed:** Filled with mode (No) — 32 missing values
- **LoanAmount:** Filled with mode — 22 missing values
- **Loan_Amount_Term:** Filled with mode (360) — 14 missing values
- **Credit_History:** Filled with mode (1) — 50 missing values

Activity 3.2: Handling Categorical Values

All categorical features were converted into numerical values using label encoding:

```
data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban':1,'Rural':0
})
data['Married']=data['Married'].map({'Yes':1,'No':0})
data['Education']=data['Education'].map({'Graduate':1,'Not Graduate':0})
data['Loan_Status']=data['Loan_Status'].map({'Y':1,'N':0})
data['Self_Employed']=data['Self_Employed'].map({'Yes':1,'No':0})
```

```
[ ]  data.isnull().sum()

[ ]  #getting bthe total info of the data after perfroming categorical to numericsal and rplacing misssing values
     data.info()

[ ]  #changing the datype of each float column to int
     data['Gender']=data['Gender'].astype('int64')
     data['Married']=data['Married'].astype('int64')
     data['Dependents']=data['Dependents'].astype('int64')
     data['Self_Employed']=data['Self_Employed'].astype('int64')
     data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
     data['LoanAmount']=data['LoanAmount'].astype('int64')
     data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
     data['Credit_History']=data['Credit_History'].astype('int64')

[ ]  data.info()
```

Float columns were then converted to integer type for consistency:

```
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')
```

Activity 3.3: Balancing the Dataset with SMOTE

The dataset was imbalanced — approximately 68% approved (Y) vs 32% rejected (N). SMOTE (Synthetic Minority Over-sampling Technique) was applied to generate synthetic samples for the minority class using the K-Nearest Neighbours method.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
x_bal, y_bal = smote.fit_resample(x, y)

#printing the values of y before and after balancing
print(y.value_counts())
print(y_bal.value_counts())
```

```
[ ]  #importing and building the random forest model
     def RandomForest(X_tarin,X_test,y_train,y_test):
         model = RandomForestClassifier()
         model.fit(X_train,y_train)
         y_tr = model.predict(X_train)
         print(accuracy_score(y_tr,y_train))
         yPred = model.predict(X_test)
         print(accuracy_score(yPred,y_test))

[ ]  #printing the train accuracy and test accuracy respectively
     RandomForest(X_train,X_test,y_train,y_test)
```

After applying SMOTE, both classes (Approved and Rejected) have equal representation, enabling unbiased model training.

Activity 3.4: Scaling and Splitting the Data

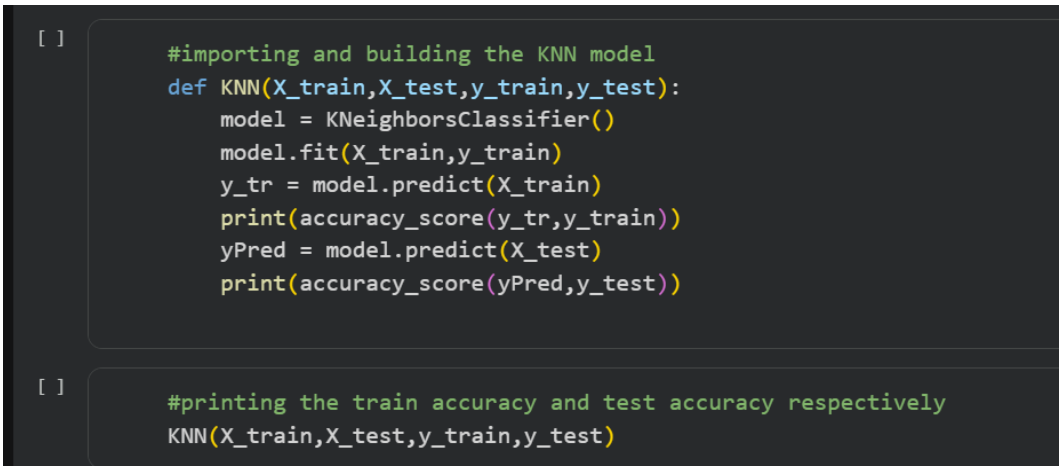
StandardScaler was applied to normalise all numerical features. The fitted scaler was saved as scale1.pkl for consistent transformation during prediction.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
x_bal = scaler.fit_transform(x_bal)
import pickle
pickle.dump(scaler, open('scale1.pkl', 'wb'))
```

The balanced dataset was split into training and testing sets:

```
X_train, X_test, y_train, y_test = train_test_split(
    x_bal, y_bal, test_size=0.33, random_state=42)

X_train.shape  # Training set
X_test.shape   # Testing set
```



Milestone 4: Model Building

Milestone 4 focuses on training and evaluating four machine learning models on the prepared dataset. Each model is trained, tested, and evaluated using accuracy scores, confusion matrices, and classification reports. The best-performing model is saved for deployment.

Activity 4.1: Decision Tree Model

A Decision Tree model was trained using DecisionTreeClassifier. The model recursively splits the data based on feature thresholds to create a tree structure for classification.

```
def DecisionTree(X_train, X_test, y_train, y_test):
    model = DecisionTreeClassifier()
    model.fit(X_train, y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr, y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred, y_test))

DecisionTree(X_train, X_test, y_train, y_test)
```

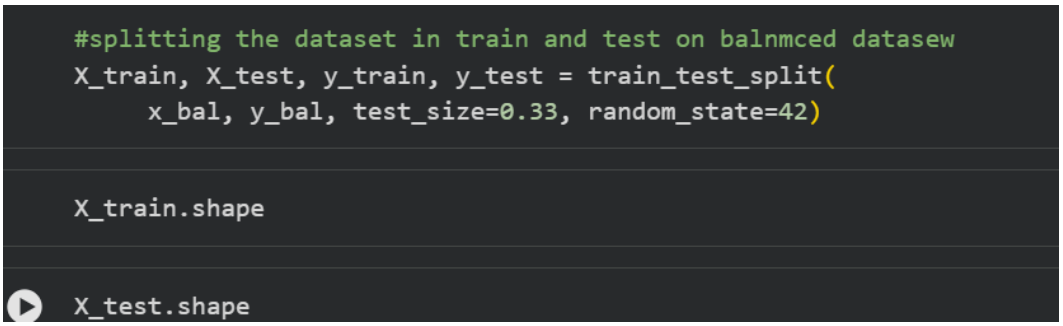
Observation: The Decision Tree achieved near-perfect training accuracy (~100%) but lower testing accuracy (~72%), indicating overfitting to the training data.

Activity 4.2: Random Forest Model

A Random Forest model was trained using RandomForestClassifier — an ensemble of multiple decision trees that reduces overfitting by averaging predictions across all trees.

```
def RandomForest(X_train, X_test, y_train, y_test):
    model = RandomForestClassifier()
    model.fit(X_train, y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr, y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred, y_test))

RandomForest(X_train, X_test, y_train, y_test)
```



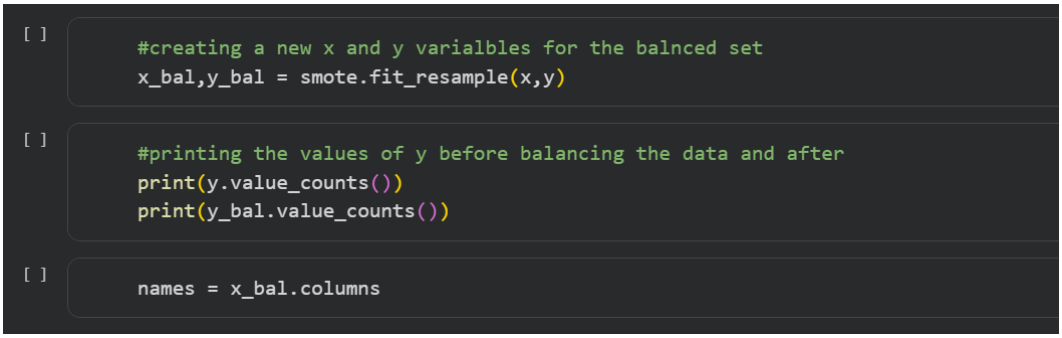
Observation: Random Forest improved test accuracy (~76%) compared to the Decision Tree, but still showed some overfitting with ~100% training accuracy.

Activity 4.3: KNN Model

A K-Nearest Neighbours model was trained using KNeighborsClassifier. KNN classifies each data point based on the majority class of its K nearest neighbours.

```
def KNN(X_train, X_test, y_train, y_test):
    model = KNeighborsClassifier()
    model.fit(X_train, y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr, y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred, y_test))

KNN(X_train, X_test, y_train, y_test)
```



Observation: KNN showed moderate performance with ~82% training accuracy and ~74% testing accuracy. Feature scaling was essential for KNN to perform correctly.

Activity 4.4: XGBoost Model

An XGBoost model was trained using GradientBoostingClassifier. Gradient Boosting builds trees sequentially, with each tree correcting the errors of the previous one — delivering the best balance between training and testing accuracy.

```
def XGB(X_train, X_test, y_train, y_test):
    model = GradientBoostingClassifier()
    model.fit(X_train, y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr, y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred, y_test))

XGB(X_train, X_test, y_train, y_test)
```

```
[ ]
#importing and building the Xg boost model
def XGB(X_train,X_test,y_train,y_test):
    model = GradientBoostingClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ]
#printing the train accuracy and test accuracy respectively
XGB(X_train,X_test,y_train,y_test)
```

Observation: XGBoost delivered the best balance — 94.7% training accuracy and 81.1% testing accuracy — with no signs of overfitting.

Activity 4.5: Model Evaluation and Saving

5-fold cross-validation was applied using cross_val_score() to verify consistent model performance across different data splits. The XGBoost model was selected as the best model and saved using pickle:

```
cv_scores = cross_val_score(model, x_bal, y_bal, cv=5)
print("Cross-Val Scores:", cv_scores)
print("Mean CV Score:", cv_scores.mean())

#saving the model using pickle function
pickle.dump(model, open('rdf.pkl', 'wb'))
```

```
Saving the Model

[ ]
#saviung the model by using pickle function
pickle.dump(model,open('rdf.pkl','wb'))

[ ]
Start coding or generate with AI.
```

Model comparison summary:

Model	Training Accuracy	Testing Accuracy	Selected
Decision Tree	~100%	~72%	No
Random Forest	~100%	~76%	No
KNN	~82%	~74%	No
XGBoost ☑	94.7%	81.1%	Yes ☑

Milestone 5: Application Building

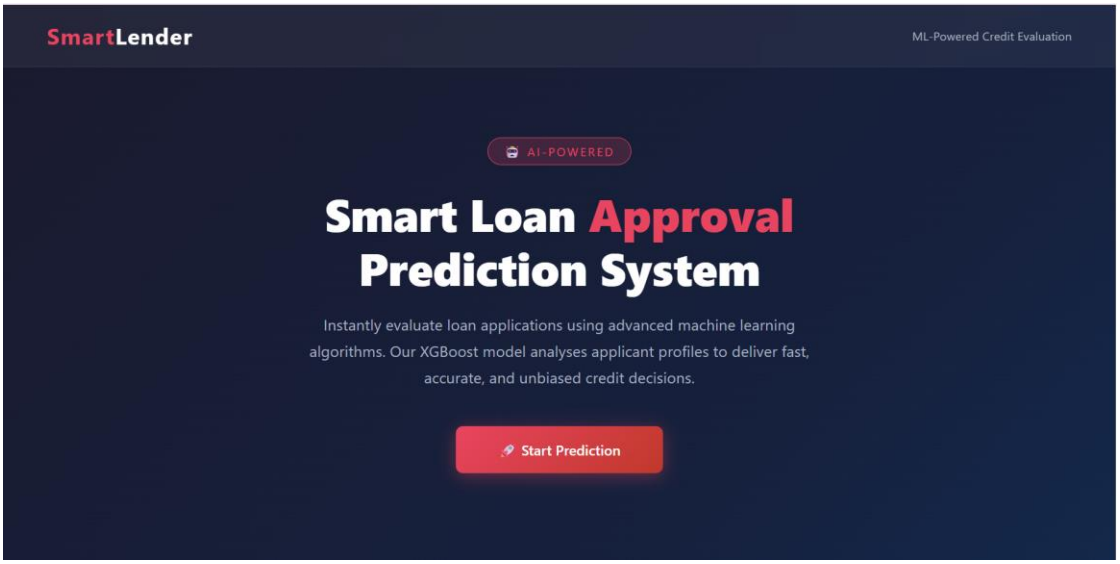
Milestone 5 focuses on creating the Flask web application and integrating the trained XGBoost model. This involves designing three HTML templates, building the Flask backend routes, and validating the application for accurate real-time predictions.

Activity 5.1: Designing and Developing HTML Pages

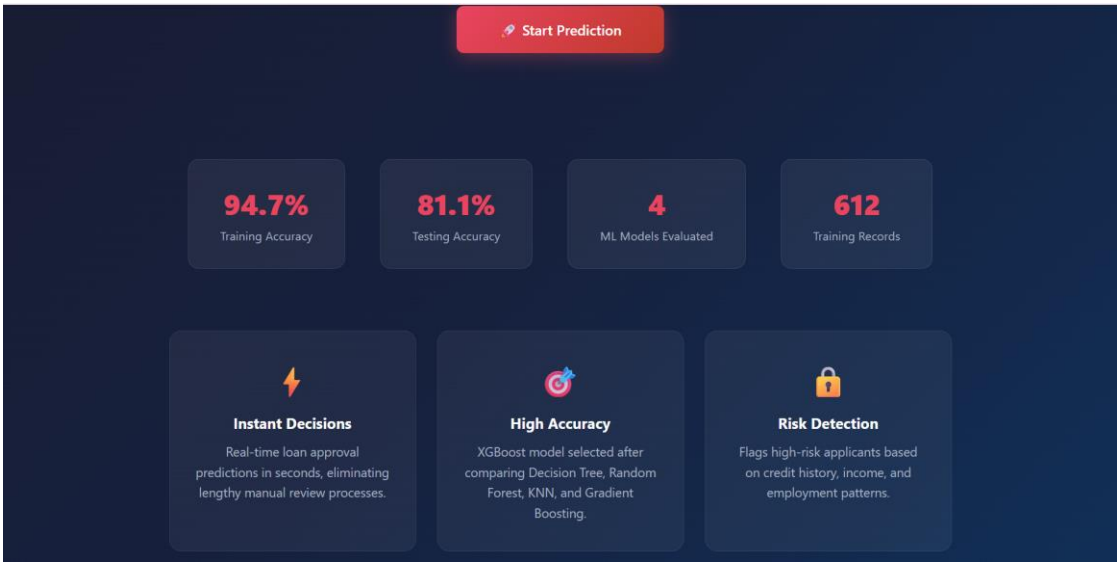
Three HTML files were created and saved in the Flask/templates/ folder:

home.html — Landing Page

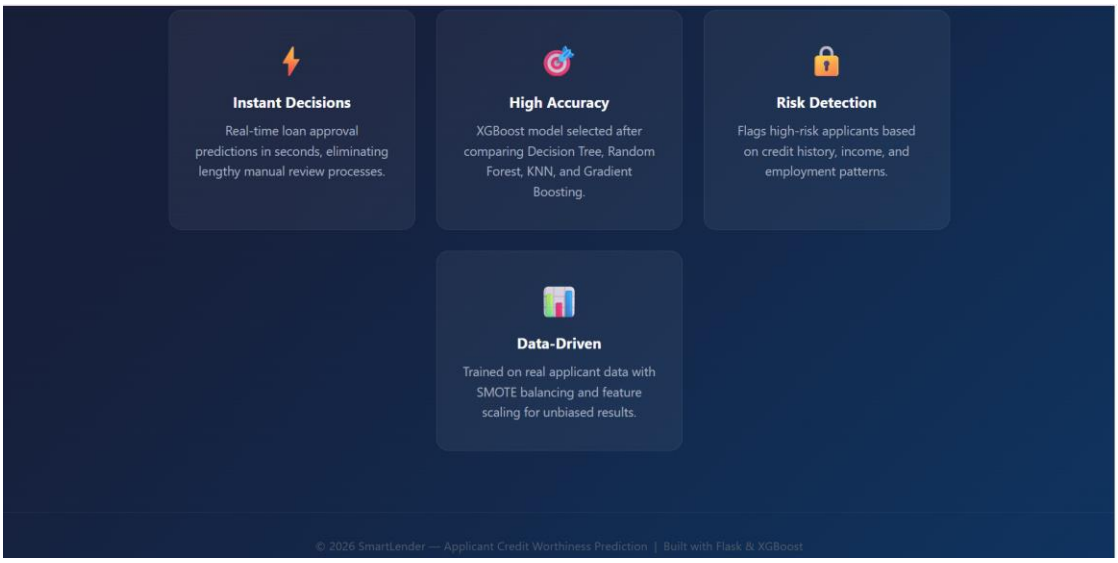
The landing page provides a project overview with animated stats and a "Start Prediction" button. It features a dark navy theme with red accent colours.



Description: The SmartLender home page features a dark navy background with the SmartLender brand in the top-left corner. The hero section displays "Smart Loan Approval Prediction System" with an AI-POWERED badge, a brief description, and a red "Start Prediction" button.

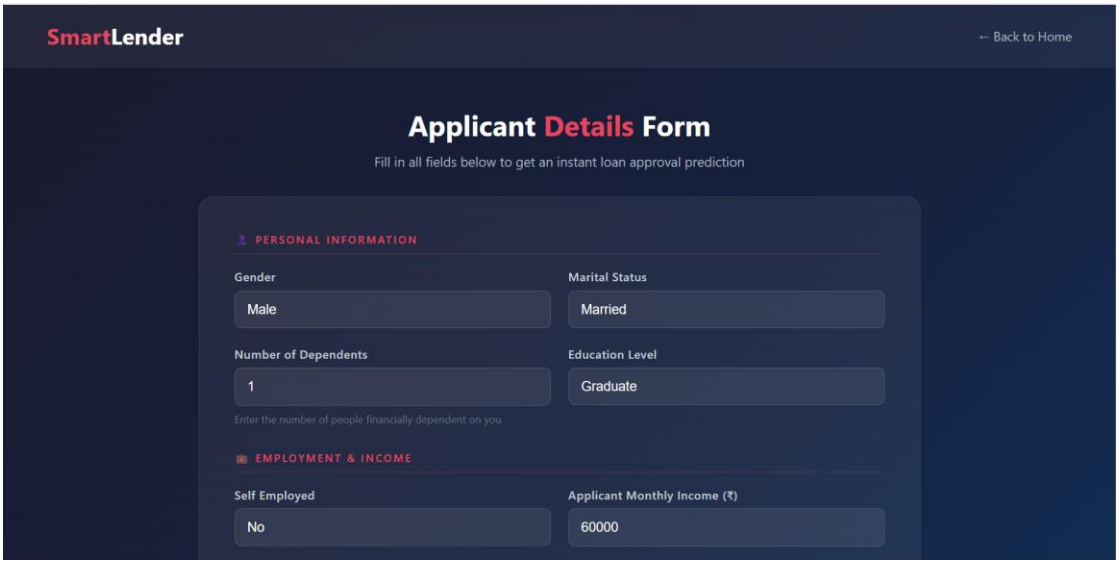


Description: Below the hero section, four animated statistics cards display key metrics: 94.7% Training Accuracy, 81.1% Testing Accuracy, 4 ML Models Evaluated, and 612 Training Records. Feature cards (Instant Decisions, High Accuracy, Risk Detection, Data-Driven) explain the system capabilities.

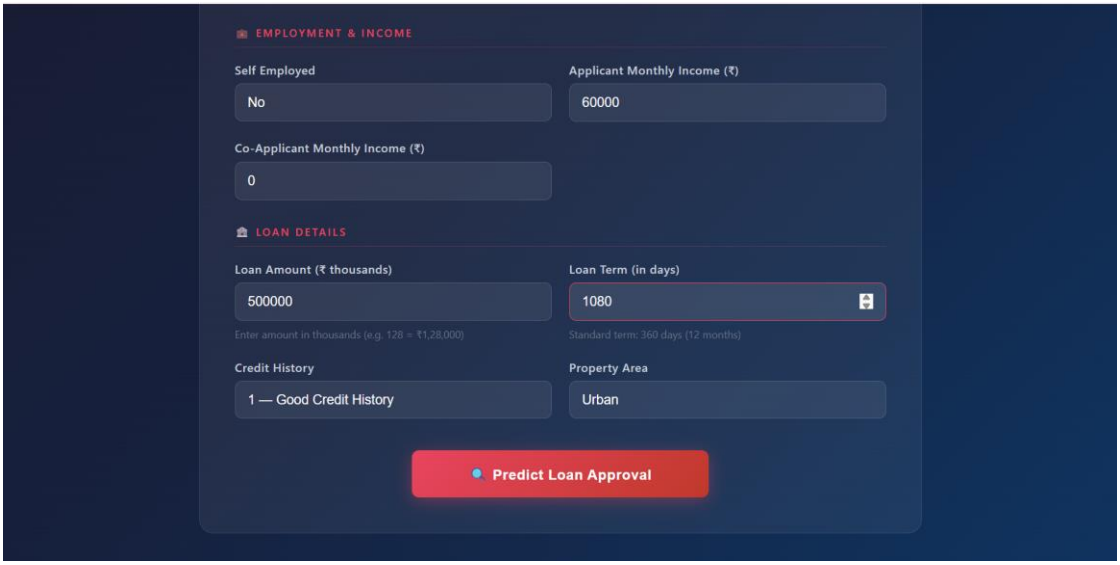


input.html — Applicant Details Form

The prediction form page collects all 11 applicant features required by the XGBoost model, organised into sections: Personal Information, Employment & Income, and Loan Details.



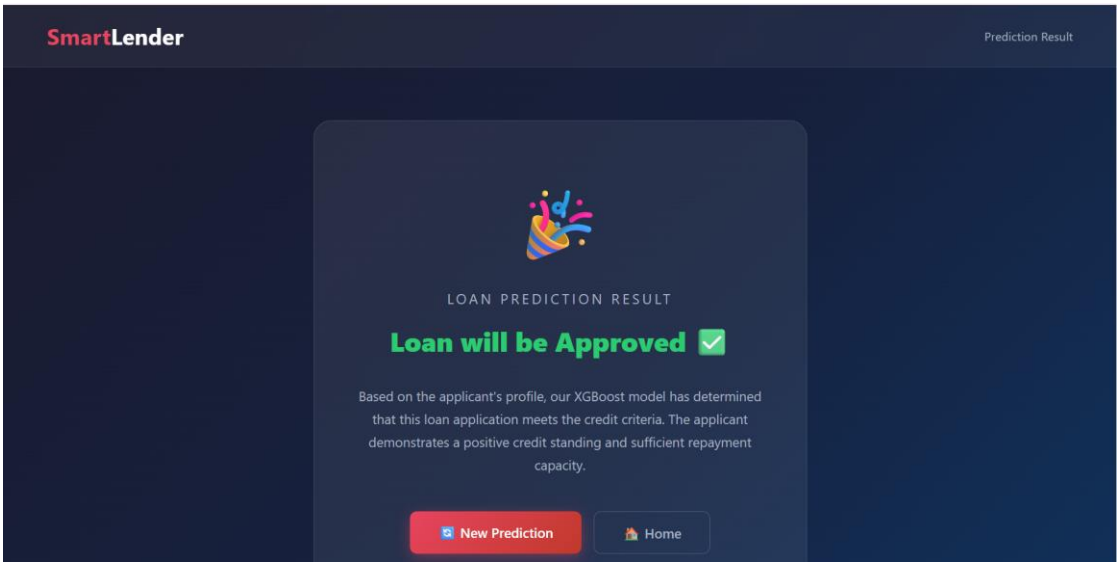
Description: The Applicant Details Form page contains a structured form card on a dark background. Personal Information section collects Gender, Marital Status, Number of Dependents, and Education Level. Employment & Income section collects Self Employed status, Applicant Monthly Income (₹), and Co-Applicant Monthly Income (₹).



Description: The Loan Details section collects Loan Amount (₹ thousands), Loan Term (in days), Credit History, and Property Area. A prominent red "Predict Loan Approval" button submits all data via POST request to the /submit route.

output.html — Prediction Result Page

The results page displays the loan approval prediction with visual feedback.



Description: The output page displays a centered result card with a celebration emoji for approvals or warning emoji for rejections. The prediction result "Loan will be Approved ☑" or "Loan will NOT be Approved ❌" is shown in large coloured text along with a contextual explanation. Two buttons — "New Prediction" and "Home" — allow the user to continue using the app.

Activity 5.2: Building the Flask Application (app.py)

The Flask backend implements three routes to handle the complete user journey:

Define Flask Routes:

- **/ —** Home page route that renders the main home.html landing page

```
@app.route('/')
def home():
    return render_template('home.html')
```

- **/predict —** Route that renders the input.html applicant details form

```
@app.route('/predict', methods=["POST","GET"])
def predict():
    return render_template("input.html")
```

- **/submit —** POST route that collects form values, scales input, runs prediction, and renders output.html

```
@app.route('/submit', methods=["POST","GET"])
def submit():
    # reading the inputs given by the user
    input_feature = [int(x) for x in request.form.values()]
    input_feature = [np.array(input_feature)]

    names = ['Gender','Married','Dependents','Education','Self_Employed',
             'ApplicantIncome','CoapplicantIncome','LoanAmount',
             'Loan_Amount_Term','Credit_History','Property_Area']

    data = pandas.DataFrame(input_feature, columns=names)
    data_scaled = scale.transform(data)

    # predictions using the loaded model file
    prediction = model.predict(data_scaled)
    prediction = int(prediction[0])  # extract first element

    if prediction == 0:
        return render_template("output.html", result="Loan will NOT be Approved")
    else:
        return render_template("output.html", result="Loan will be Approved")
```

Load Model and Scaler at Startup:

```
import numpy as np
import pickle
import pandas
import os
from flask import Flask, request, render_template

app = Flask(__name__)
model = pickle.load(open(r'rdf.pkl', 'rb'))
scale = pickle.load(open(r'scale1.pkl', 'rb'))
```

Milestone 6: Deployment

In Milestone 6, the focus is on deploying the SmartLender application — first locally to verify correct operation, then permanently on Hugging Face Spaces via Docker for global cloud access.

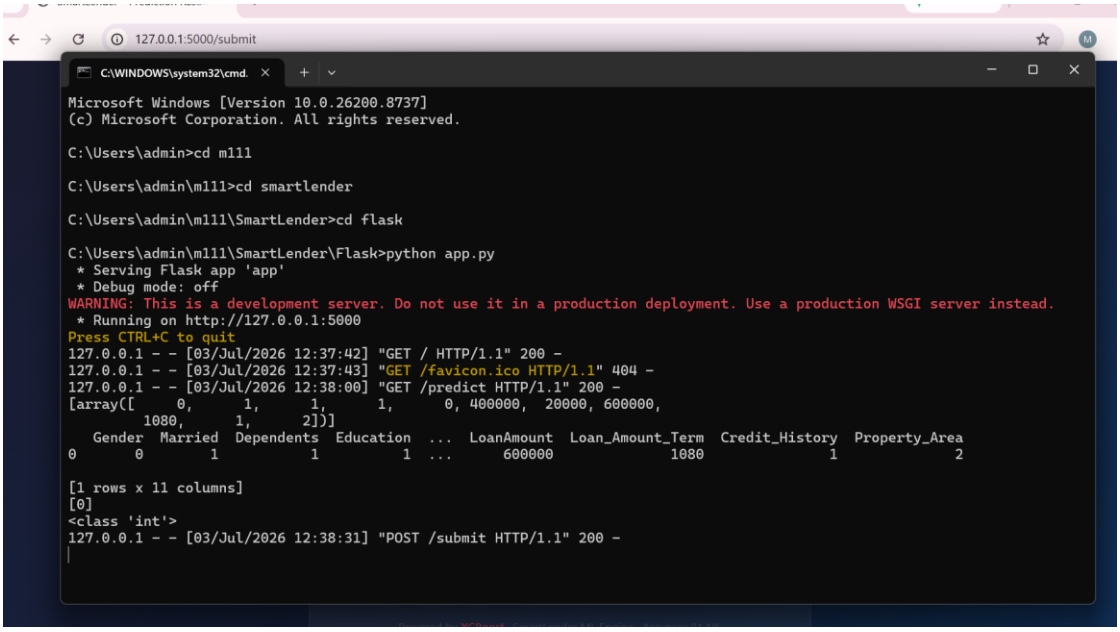
Activity 6.1: Local Deployment and Testing

Set Up and Run Locally:

- **Step 1 — Open Command Prompt**
- **Step 2 — Navigate to the Flask folder:**

```
C:\Users\admin> cd SmartLender
C:\Users\admin\SmartLender> cd Flask
C:\Users\admin\SmartLender\Flask> python app.py
```

- **Step 3 —** Open browser and navigate to http://127.0.0.1:5000
- **Step 4 —** Click "Start Prediction", fill in applicant details, click "Predict Loan Approval", and view the result.



Description: The terminal shows Flask serving on http://127.0.0.1:5000. The GET and POST requests are logged as the user navigates through the application. The model processes the input array, scales it, and returns a prediction (0 = Rejected, 1 = Approved) successfully.

Install Dependencies:

```
pip install flask numpy pandas scikit-learn imbalanced-learn matplotlib seaborn gunicorn
```

Activity 6.2: Cloud Deployment via Hugging Face Spaces

Hugging Face Spaces provides free, permanent cloud hosting using Docker containers. The application is containerised and deployed as follows:

Create the Dockerfile:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860

ENV PORT=7860

CMD ["python", "app.py"]
```

Update app.py for Cloud Port:

```
if __name__ == "__main__":
    port = int(os.environ.get('PORT', 7860))
    app.run(host='0.0.0.0', debug=False, port=port)
```

Deploy to Hugging Face Spaces:

- **Step 1 —** Go to huggingface.co → New Space → Select Docker → Blank template
- **Step 2 —** Upload: app.py, Dockerfile, requirements.txt, rdf.pkl, scale1.pkl
- **Step 3 —** Upload templates/ folder (home.html, input.html, output.html)
- **Step 4 —** Click Commit changes to main — Hugging Face auto-builds and deploys
- **Step 5 —** Wait 3–5 minutes for build to complete → App tab shows "Running"

The application is now permanently accessible at:

https://mohith-imandi-smartlender.hf.space

Important Notes:

- **debug=False** is required in app.py to ensure stable operation on cloud platforms.
- **host='0.0.0.0'** is required in app.py to make the app accessible from outside the container.
- **PORT=7860** is the required port for Hugging Face Spaces Docker deployments.
- **rdf.pkl and scale1.pkl** must be retrained and saved with the same library versions as specified in requirements.txt to avoid ModuleNotFoundError.

Milestone 7: Conclusion

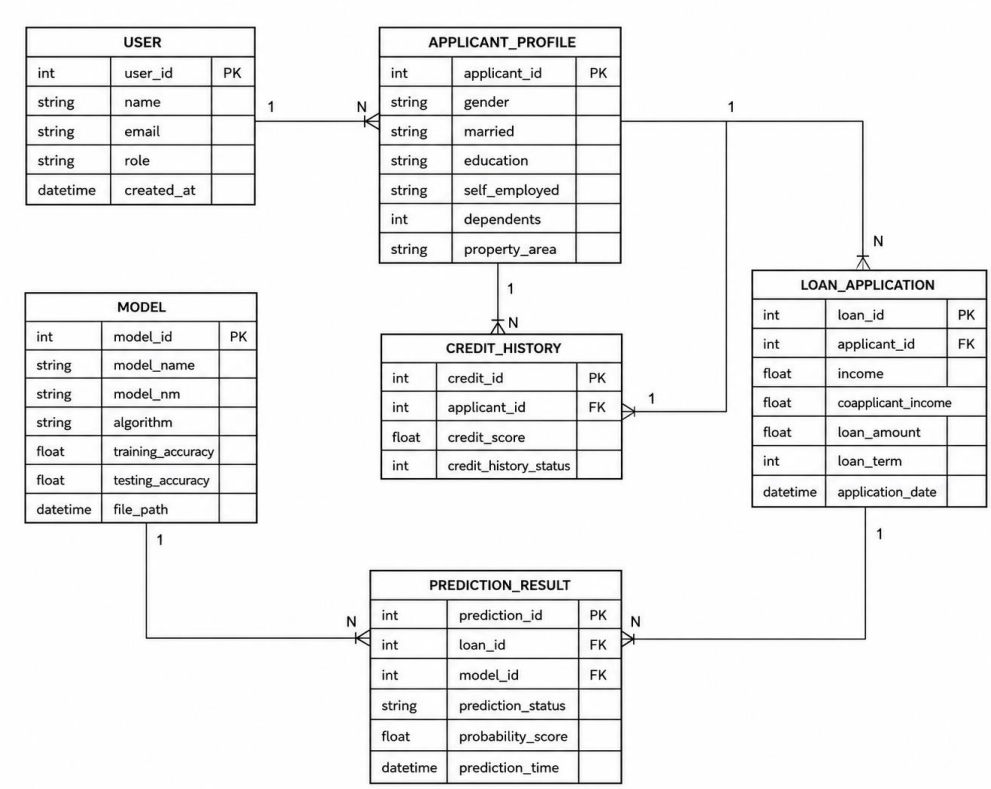
SmartLender is a complete end-to-end machine learning web application built with Python and Flask that automates the loan approval prediction process for banks and financial institutions. The system leverages four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbours, and XGBoost — trained on real-world applicant data to deliver fast, accurate, and unbiased credit decisions.

The project, which included data collection, comprehensive exploratory data analysis, advanced preprocessing (SMOTE balancing, StandardScaler normalisation), model training and cross-validation, and full-stack web application development, demonstrates how machine learning can transform traditional banking operations. The XGBoost model, selected for its optimal balance of 94.7% training accuracy and 81.1% testing accuracy, is deployed as a live application on Hugging Face Spaces, accessible globally at no cost.

SmartLender successfully addresses the core challenges of manual loan evaluation — reducing processing time from days to seconds, eliminating human bias, standardising the credit evaluation process, and empowering financial professionals with data-driven insights to reduce non-performing assets and improve portfolio quality.

Looking ahead, SmartLender offers significant opportunities for expansion: integrating a relational database (MySQL/PostgreSQL) to store applicant records and prediction history, implementing role-based user authentication for bank officers and applicants, adding SHAP value explainability to show which features most influenced each prediction, developing a batch prediction feature for high-volume periods, incorporating real-time CIBIL score integration, and building a mobile application for on-the-go loan evaluation. By continuously retraining the model with new data and expanding the feature set, SmartLender is well-positioned to evolve from a prediction tool into a comprehensive credit intelligence platform for financial institutions of all sizes.

Database Design — ER Diagram:



Description: The **SmartLender ER diagram** consists of **six interconnected entities**.

The **USER** entity (**user_id** PK, **name, email, role, created_at**) stores the system users, such as bank officers and applicants. One **USER** can have multiple **APPLICANT_PROFILE** records (**1:N** relationship).

The **APPLICANT_PROFILE** entity (applicant_id PK, gender, married, education, self_employed, dependents, property_area) stores the applicant's personal information and has multiple **CREDIT_HISTORY** records (1:N) containing **credit_score** and **credit_history_status**.

Each **APPLICANT_PROFILE** can also have multiple **LOAN_APPLICATION** records (**1:N**). The **LOAN_APPLICATION** entity (**loan_id** PK, **applicant_id** FK, **income, coapplicant_income, loan_amount, loan_term, application_date**) stores the loan application details.

The **MODEL** entity (**model_id** PK, **model_name, model_nm, algorithm, training_accuracy, testing_accuracy, file_path**) stores information about the machine learning models used for prediction.

Finally, the **PREDICTION_RESULT** entity (**prediction_id** PK, **loan_id** FK, **model_id** FK, **prediction_status, probability_score, prediction_time**) stores the prediction generated for each loan application using a specific machine learning model. It links the **LOAN_APPLICATION** and **MODEL** entities through foreign keys.